



ADR-004: Persistence — PostgreSQL document + indexed-columns pattern, with an in-memory adapter that behaves the same

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

Document	Version	Date	Owner	Status
ADR-004	1.0	07 October 2026	iorta TechNXT Architecture	Accepted

- **Related:** [ADR-005](#), [ADR-006](#), [ADR-008](#), [Data Architecture](#)

Context and problem statement

The aggregates are document-shaped and evolve quickly. A golden profile carries nested vehicle, policy, engagement, channel, consent, lineage and expiry-candidate structures. Leads, quotes and voice sessions are similar. Queries filter on a small, known set of attributes: tier, score, region, expiry date, status and similar.

The platform also needs:

- ACID transactions, row locking for the outbox, PITR backups and a managed HA option acceptable to TASCO InfoSec;
- per-field encryption of PII before it reaches the database;
- a zero-dependency store for unit tests, demos and pilots.

Decision drivers

- Agility of the document schema, combined with indexed, typed filtering.
- Injection-proof querying by construction.
- One encryption and indexing code path for every store.
- Managed Postgres is available on every target cloud and on Railway.

Considered options

1. PostgreSQL: one table per collection with `id`, `version`, `data jsonb`, typed indexed columns extracted from the document, `created_at`, `updated_at`. An in-memory adapter implements the same port and shares the codec.
2. A fully normalised relational schema with an ORM.
3. A document database (MongoDB/DocumentDB).
4. JSONB with GIN indexes only (no extracted columns).

Decision outcome

Chosen option: **1.**

- **Collection registry.** `src/adapters/persistence/schema.js` is the single source of truth. It holds 19 collections, and each declares:
 - `indexes: {column: {path, type}};`
 - `pii: [fields]` (encrypted);
 - `blind: {column: field}` (HMAC blind index).

Collection and column names must match `^[a-z][a-z0-9_]*$` (checked at module load).

- **Migrations.** Numbered SQL files in `db/migrations/` (`001_init.sql`, `002_integrity_hardening.sql`). The collection DDL is generated by `schemaSql.js`, and a schema test keeps the migration and the registry in step. Applied migrations are recorded in `schema_migrations`; each runs in its own transaction, and

`migrate()` holds a PostgreSQL advisory lock so concurrent runners cannot race (`postgresStore.migrate`). Never edit an applied migration; add a new numbered file.

- **Codec** (`codec.js`). Shared by both stores. It encrypts `pii` fields, computes blind indexes, and extracts index column values from document paths. Indexed columns are therefore never PII (by convention: review new indexes).
- **Query safety**. Every statement uses `$n` bind parameters. Identifiers come only from the registry. `assertColumns()` (`query.js`) allows filtering and sorting only on declared columns plus `id`, `created_at` and `updated_at`. Operators are limited to `in`, `gte`, `lte`, `gt`, `lt`, `ne`, `isNull`. `LIMIT` is capped at 5,000.
- **Optimistic concurrency**. `update()` increments `version` and, when the caller passes the version it read, adds `AND version = $n`. A mismatch raises `409 CONFLICT`. `upsert()` is last-writer-wins by design (used for idempotent rebuilds).
- **In-memory adapter** (`memoryStore.js`). Implements the same port with identical semantics (codec, `matches()`, version checks, audit chain). `transaction(fn)` simply runs `fn(this)`, and `claimEvents` marks rows as processing.
- **Connection**. `pg.Pool` with `max = DB_POOL_MAX` (default 10), `statement_timeout 15 s`, idle timeout 30 s, and TLS with `rejectUnauthorized: true` when `DATABASE_SSL` is set (the default in production).

Consequences

- Good: schema evolution inside `data` needs no migration. A new filter column needs a new numbered migration plus a registry entry.
- Good: SQL injection is structurally excluded. No dynamic SQL is built from input.
- Good: tests run against memory, and `test:pg` runs the same suites against real Postgres in CI.
- Bad: the data is duplicated (the `data` jsonb plus extracted columns). Columns are recomputed on every write, so they cannot drift.
- Bad: there are no foreign keys between collections, so referential integrity is enforced in services and checked by `opsService.reconcile`. For an analytical relational view, see the [Data Architecture](#).
- Bad / gaps:
 - `store.transaction()` is used for rule activation only. Purchase is protected by an atomic quote claim (optimistic version check) and saga compensation rather than one transaction; erase, and state change + outbox publish ([ADR-005](#)), are still separate writes. **Fix in phase 2:** pass the transactional store into those use cases.
 - Migrations still run preferably from the single Kubernetes migration Job (`MIGRATE_ON_START=false` on replicas); the advisory lock makes an accidental concurrent start safe.
 - The full-table `OFFSET` paging used by batch jobs must move to keyset paging at 6 M rows.

Pros and cons of the options

Normalised schema + ORM. Strong integrity. However, every attribute change becomes a heavy migration, the ORM adds dependencies, and encrypted fields complicate ORM mapping.

MongoDB. A natural document fit. However, it would add a second operational database (Postgres is still needed for the outbox, audit and BI), and InfoSec familiarity is lower.

JSONB + GIN only. Simpler. However, it weakens typed comparisons (date and int ranges) and planner statistics, and it makes the "only declared fields are filterable" guard harder to enforce.